

QCMS Complete 131-Table Visual Architecture

White Background High-Resolution Database Diagrams (All 131 Tables as Cylinder Nodes) & Data Flow Specifications

131 Database Tables

214 Foreign Key Connections

10 Enterprise Subsystems

PostgreSQL 16 Multi-Tenant

131

TOTAL DATABASE TABLES

214

FOREIGN KEY CONNECTIONS

10

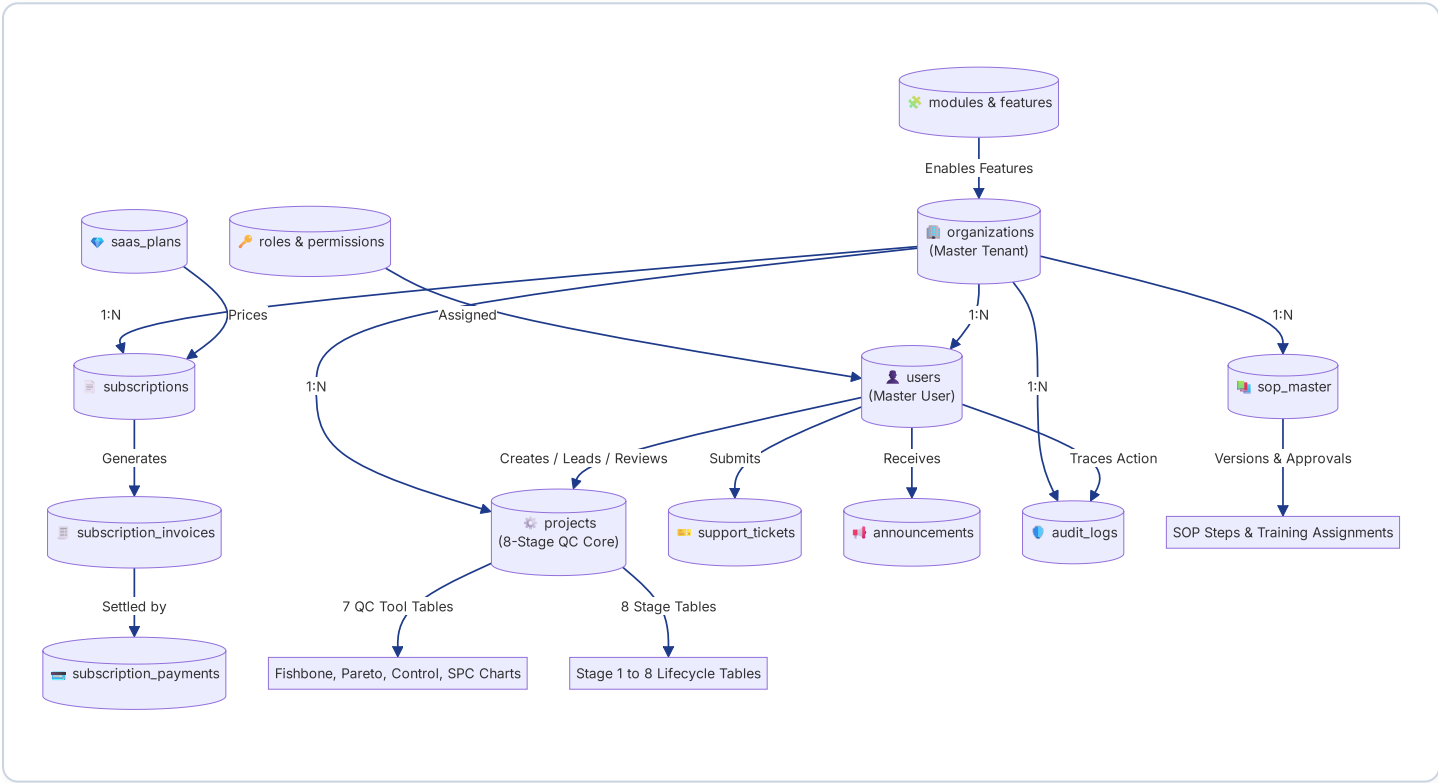
DOMAIN SUBSYSTEMS

100%

EVERY TABLE VISUALIZED

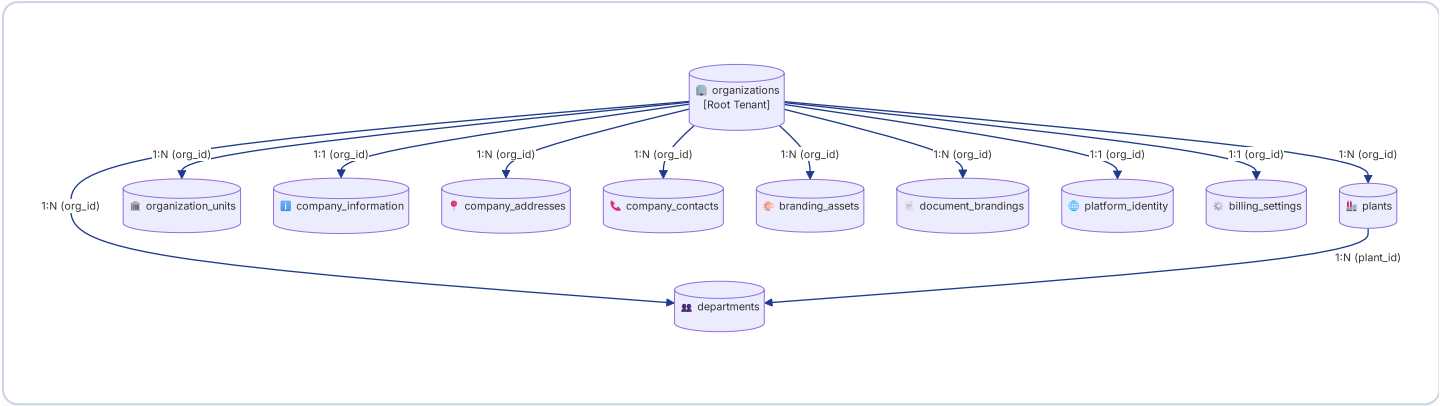
1. Master Global System Architecture (Enterprise Topology)

High-level architectural topology showing how the core root database tables connect through primary 1:N foreign key chains:



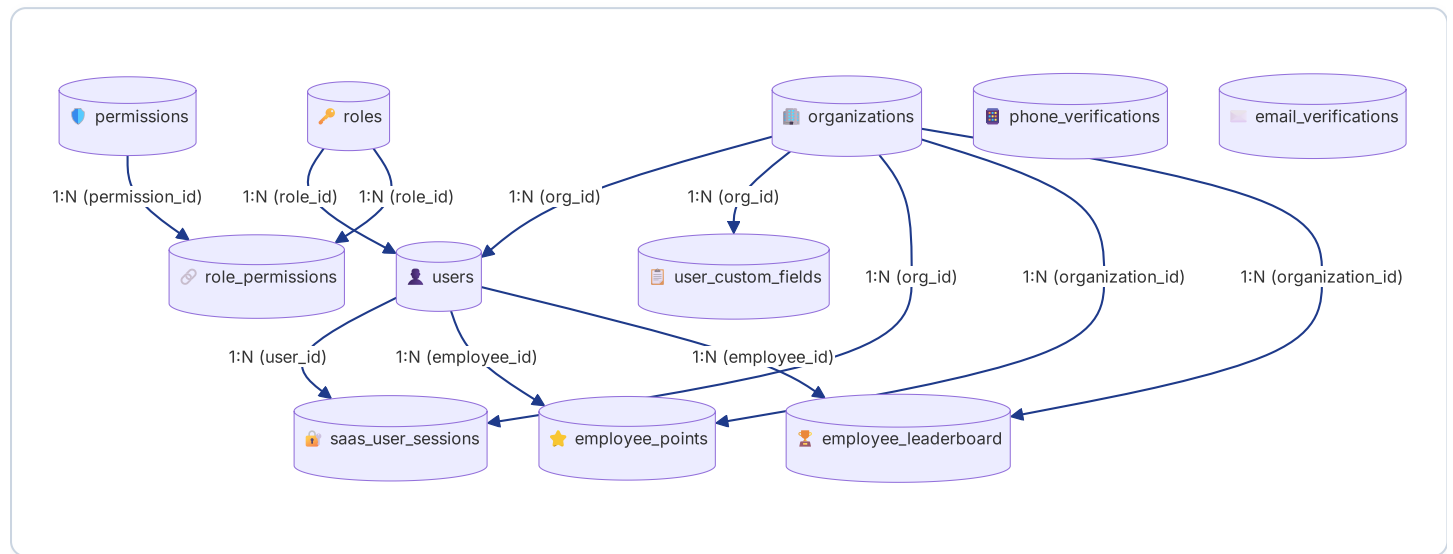
2. Domain 1: Multi-Tenancy & Corporate Structure (All 11 Tables)

Visual diagram of all 11 tables in Domain 1 showing company profile, plants, departments, organization units, branding assets, and white-label identities.



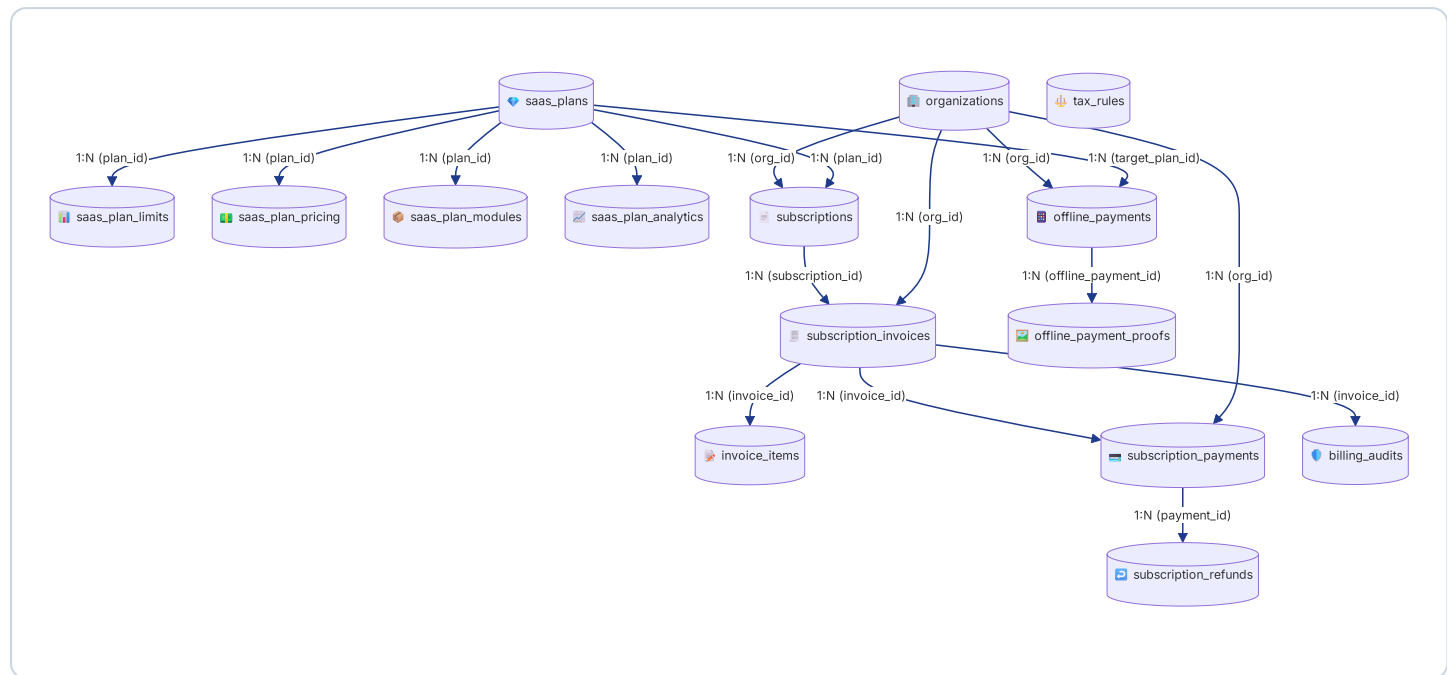
3. Domain 2: Identity, Access Control & RBAC (All 10 Tables)

Visual diagram of all 10 tables governing authentication, 6 system roles, permissions mapping, user sessions, custom fields, and gamification points.



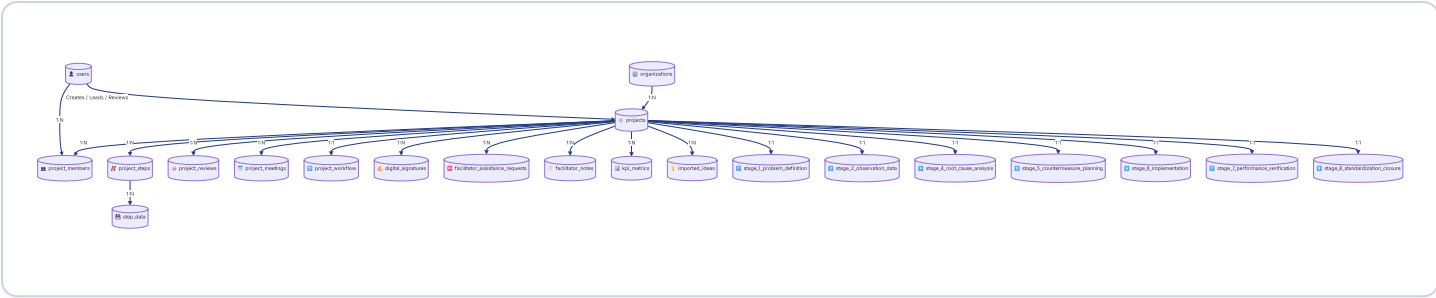
4. Domain 3: SaaS Plans, Subscriptions & PAYG Financials (All 14 Tables)

Visual diagram of all 14 tables handling subscription tiers, pricing matrices, metered PAYG billing, invoices, payments, refunds, and offline QR proofs.



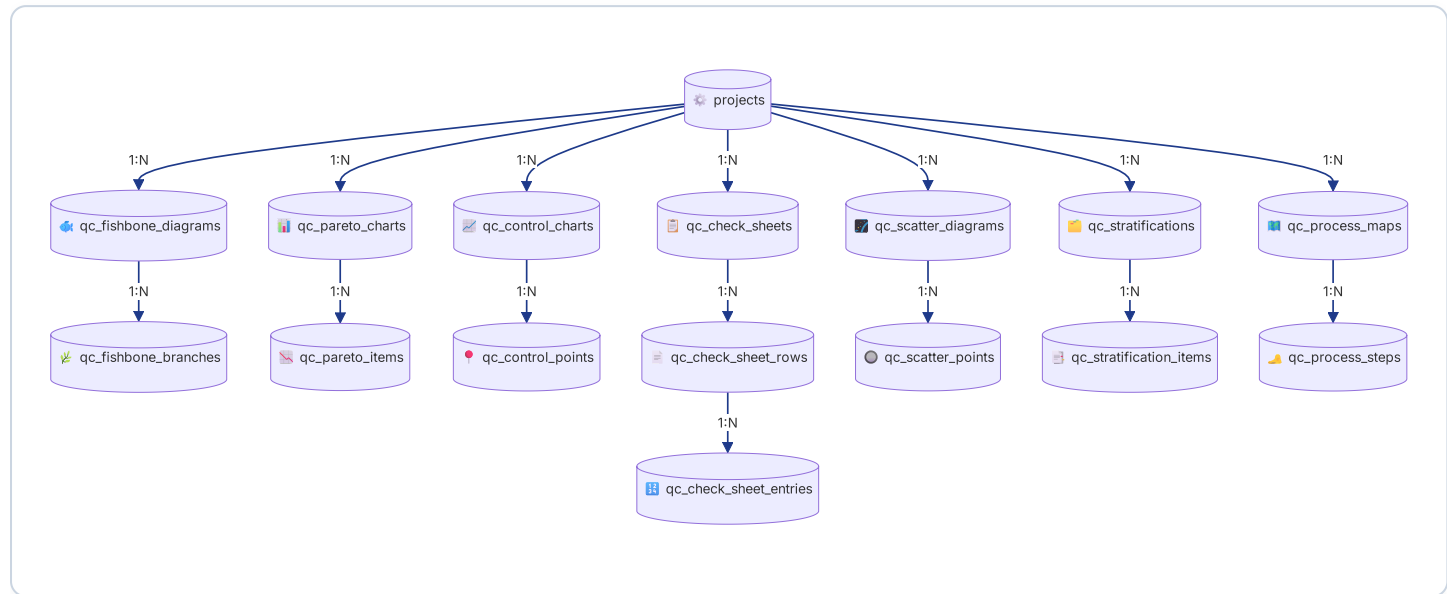
5. Domain 4: 8-Stage QC Quality Circle Workflow Engine (All 19 Tables)

Visual diagram of all 19 tables in the core 8D Quality Circle DMAIC engine, detailing project charters, members, meetings, reviews, digital signatures, and dedicated tables for Stages 1 through 8.



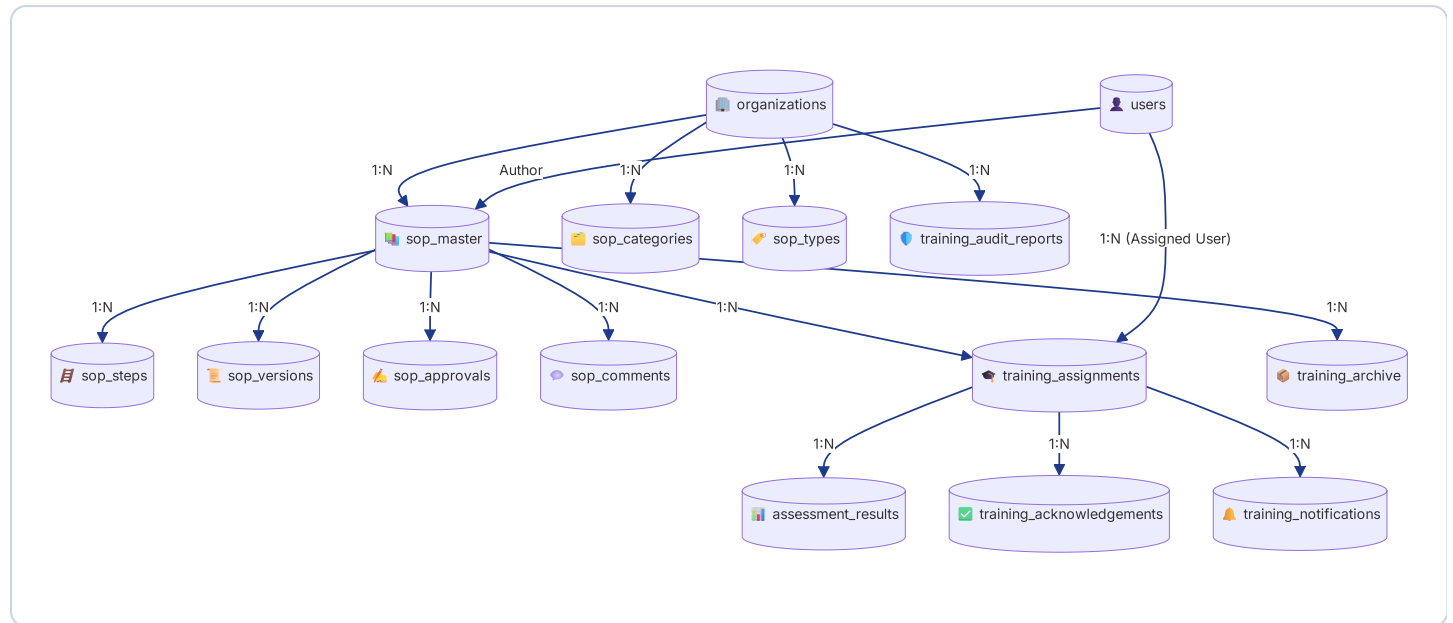
6. Domain 5: 7 QC Tools & Statistical Process Control (All 15 Tables)

Visual diagram of all 15 tables capturing raw shop-floor quality data and rendering Ishikawa 6M fishbone branches, Pareto items, SPC control points, check sheet entries, scatter correlations, and process map steps.



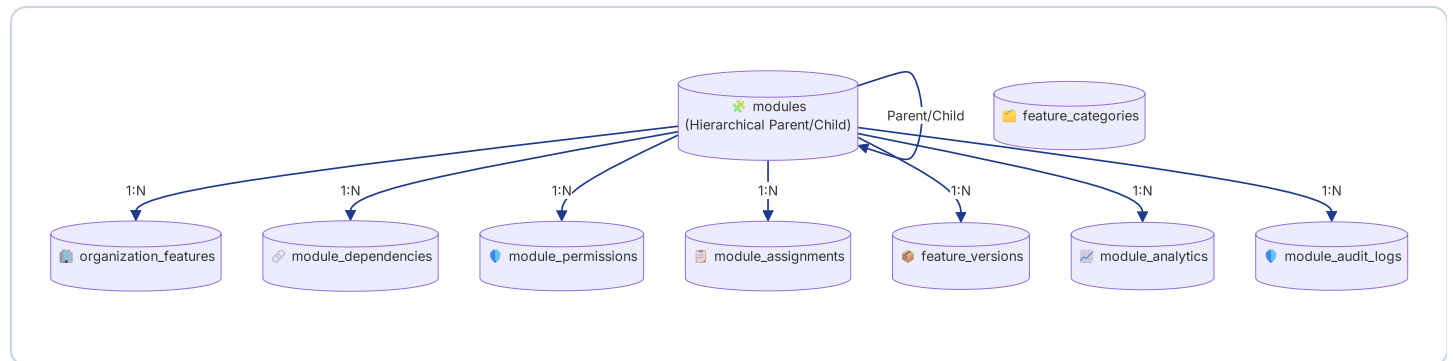
7. Domain 6: Standard Operating Procedures (SOP) & Training (All 13 Tables)

Visual diagram of all 13 tables standardizing operating procedures, revision histories, multi-tier approvals, mandatory employee training assignments, and assessment results.

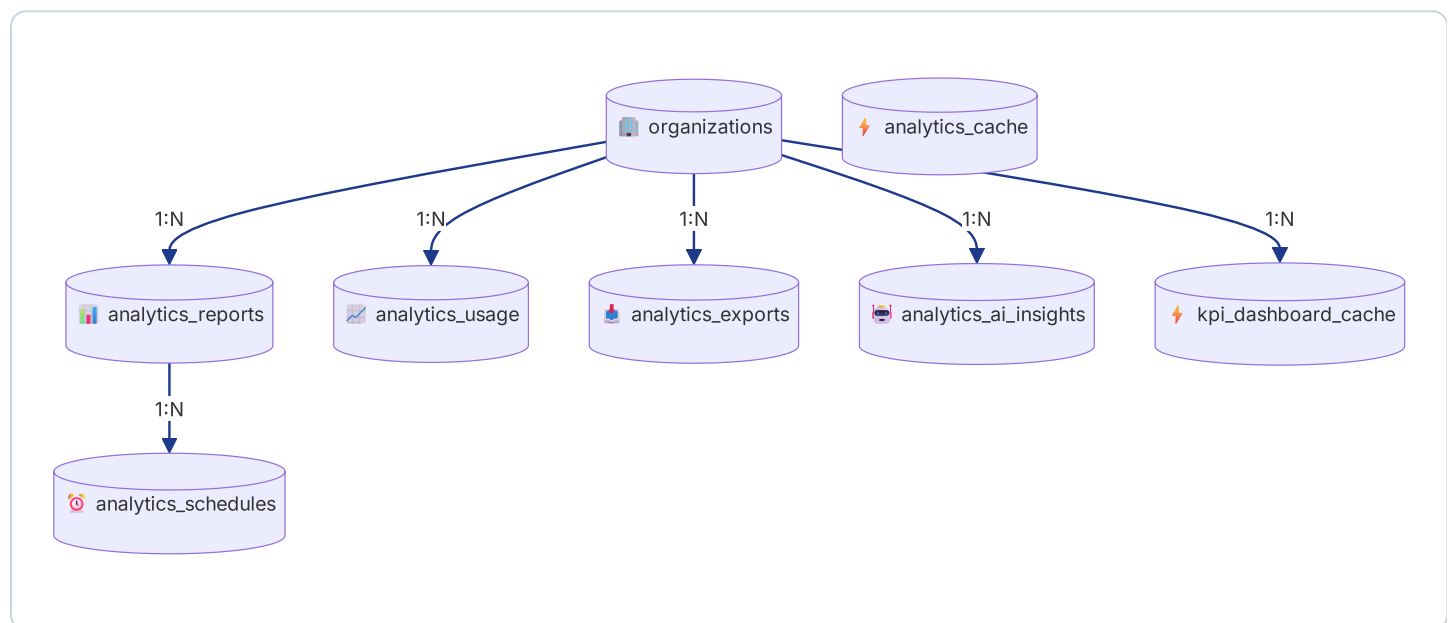


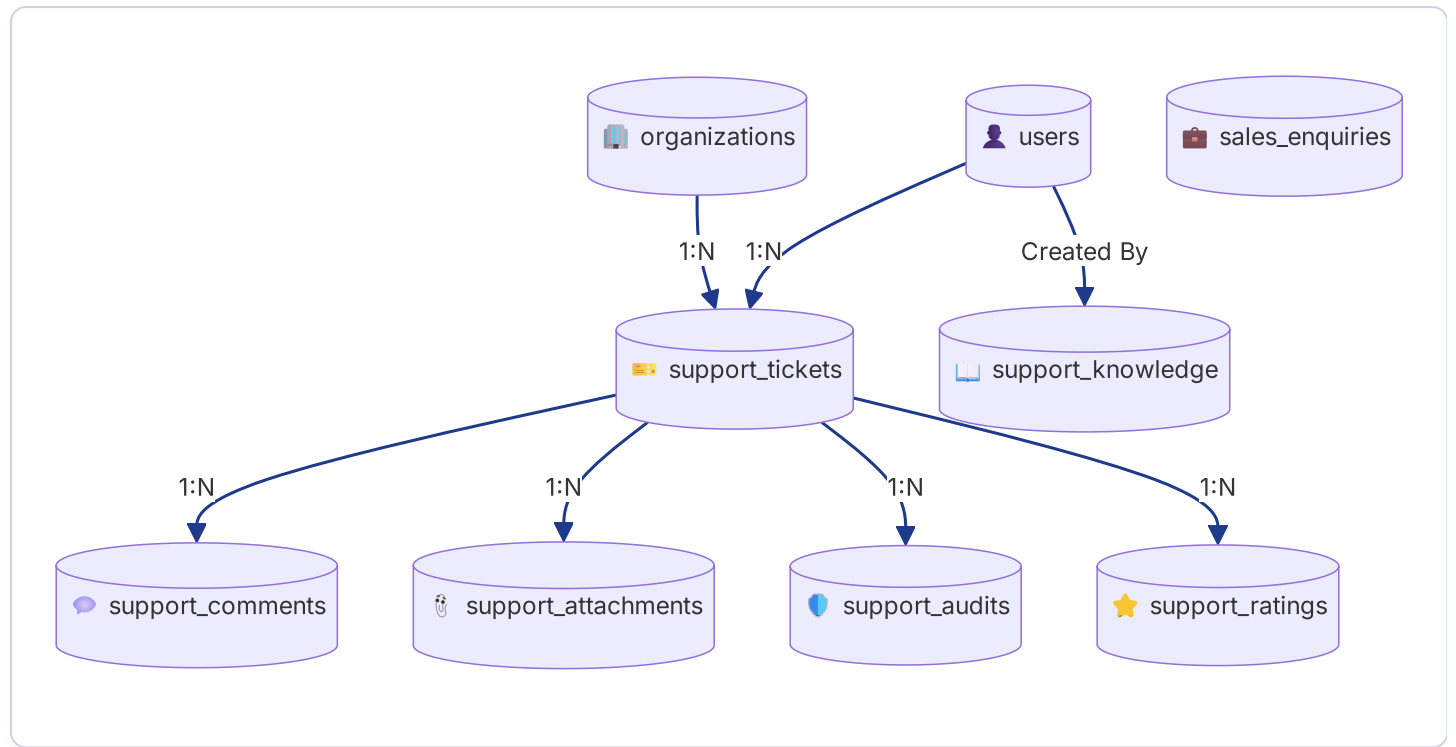
8. Domains 7, 8 & 9: Dynamic Modules, Analytics & Helpdesk Support

Domain 7: Dynamic Enterprise Modules & Feature Gating (All 9 Tables)



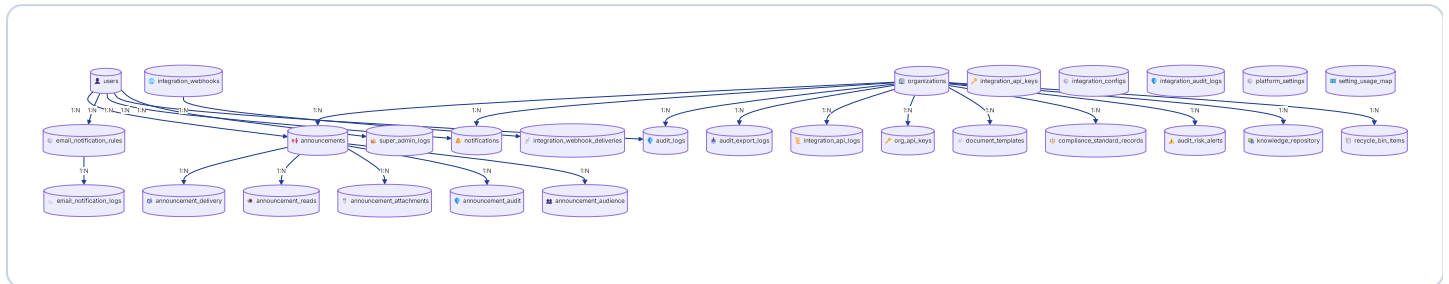
Domain 8: Analytics, AI Insights & BI Reporting (All 7 Tables)



Domain 9: Helpdesk Support, SLAs & Sales Prospect Intake (All 7 Tables)

9. Domain 10: Communications, Announcements, Integrations & Governance (All 26 Tables)

Visual diagram of all 26 tables handling tenant broadcasts, automated email rule dispatchers, notifications, security audit trails, API keys, webhooks, and global platform settings.



10. Exhaustive Technical Catalog of All 131 Tables (Field-by-Field & Ping-by-Ping)

Below is the complete exhaustive catalog of all 131 database tables across all 10 domains detailing primary keys, outgoing foreign keys, child references, data acquisition sources, and stored payloads.

Domain 1: Multi-Tenancy & Corporate Structure (11 Tables)

Defines enterprise boundaries, multi-plant physical locations, functional departments, corporate branding, and white-label identities.

organizations

55 Columns | PK: id

Outgoing Foreign

Keys:None (Root / Master Entity)

Child Table

References:analytics_ai_insights.org_id analytics_exports.org_id analytics_reports.org_id analytics_schedules.org_id analytics_usage.org_id announcement_delivery.org_id announcement_reads.org_id announcements.org_id +60 more

How it Gets Data:Super Admin Onboarding / Self Registration Wizard (Triggered by New tenant onboarding wizard)

What Data is Stored:Company legal name, org code, max seats, static storage quota (MB), subscription status

Key Schema

Columns:id, name, industry, admin_name, email, phone, logo_url, favicon_url, primary_color, timezone, date_format, currency, ...

plants

6 Columns | PK: id

Outgoing Foreign

Keys:org_id → organizations.id

Child Table

References:departments.plant_id users.plant_id

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, org_id, name, code, location, created_at

departments

5 Columns | PK: id

Outgoing Foreign

Keys:org_id → organizations.id plant_id → plants.id

Child Table

References:knowledge_repository.department_id projects.department_id sop_master.department_id users.department_id

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, org_id, plant_id, name, created_at

company_information

17 Columns | PK: id

Outgoing Foreign

Keys:org_id → organizations.id

Child Table

References:No direct child tables

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, org_id, legal_company_name, trading_name, company_description, gstin, pan, cin, msme, registration_number, tax_number, license_number, ...

company_addresses

11 Columns | PK: id

Outgoing Foreign

Keys:

org_id → organizations.id

Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata



Key Schema

Columns:

id, org_id, registered_office, corporate_office, billing_office, country, state, city, pin,

created_at, updated_at

company_contacts

27 Columns | PK: id

Outgoing Foreign

Keys:

org_id → organizations.id

Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata



Key Schema

Columns:

id, org_id, general_email, general_sender_name, support_email, support_sender_name,

billing_email, billing_sender_name, otp_email, otp_sender_name, contact_email, contact_sender_name, ...

branding_assets

12 Columns | PK: id

Outgoing Foreign

Keys:

org_id → organizations.id

Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata



Key Schema

Columns:

id, org_id, favicon_url, logo_url, dark_logo_url, light_logo_url, print_logo_url,

pdf_logo_url, email_logo_url, watermark_logo_url, created_at, updated_at

platform_identity

19 Columns | PK: id

Outgoing Foreign

Keys:

org_id → organizations.id

Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata



Key Schema

Columns:

id, org_id, software_name, software_short_name, software_display_name, platform_title,

platform_subtitle, tagline, version, edition, website, support_portal, ...

 **billing_settings**

7 Columns | PK: id

 **Outgoing Foreign**

Keys:

org_id → organizations.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

User Form Input via Web UI (Triggered by User Action via REST API)

 **What Data is Stored:**

Relational business entity attributes and metadata

 **Key Schema**

Columns: id, org_id, auto_collection, reminder_schedule, grace_period_days, payment_retry_attempts, updated_at

Domain 2: Identity, Access Control & RBAC (10 Tables)

Manages user credentials, role-based authorization matrices, dynamic permissions, user custom attributes, and authentication sessions.

users 27 Columns PK: id 🔗 Outgoing Foreign Keys: plant_id → plants.idrole_id → roles.idorg_id → organizations.iddepartment_id → departments.id 📄 Child Table References: analytics_exports.user_idanalytics_reports.created_by_idanalytics_usage.user_idannouncement_attachments.uploaded_byannouncement_audit.user_idannouncement_delivery.user_idannouncement_reads.user_idannouncements.created_by+61 more 📄 How it Gets Data: Admin User Form / CSV Bulk Import (Triggered by User creation or bulk CSV upload) 📦 What Data is Stored: User login credentials, bcrypt password hash, 2FA secret, plant/department assignment 📄 Key Schema Columns: id, org_id, plant_id, username, full_name, employee_id, phone, email, hashed_password, role_id, department_id, is_active, ...
roles 2 Columns PK: id 🔗 Outgoing Foreign Keys: None (Root / Master Entity) 📄 Child Table References: users.role_id 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: id, name
user_custom_fields 8 Columns PK: id 🔗 Outgoing Foreign Keys: org_id → organizations.id 📄 Child Table References: No direct child tables 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: id, org_id, field_key, display_name, is_required, is_system, data_type, created_at
saas_user_sessions 12 Columns PK: session_id 🔗 Outgoing Foreign Keys: org_id → organizations.iduser_id → users.id 📄 Child Table References: No direct child tables 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: session_id, user_id, org_id, login_time, logout_time, session_duration, device, browser, os, ip_address, location, status

 employee_points

10 Columns | PK: id

 Outgoing Foreign

Keys:

project_id → projects.id

employee_id → users.id

created_by → users.id

organization_id → organizations.id

 Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata



Key Schema

Columns:

id, employee_id, organization_id, project_id, activity_type, activity_reference_id, points, description, created_by, created_at

 employee_leaderboard

13 Columns | PK: id

 Outgoing Foreign

Keys:

employee_id → users.id

organization_id → organizations.id

 Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns:

id, employee_id, organization_id, total_points, projects_completed, projects_created, ideas_submitted, ideas_approved, knowledge_articles, meetings_attended, badges, rank, ...

 phone_verifications

6 Columns | PK: id

 Outgoing Foreign

Keys:

None (Root / Master Entity)

 Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns:

id, phone, otp, is_verified, expires_at, created_at

 email_verifications

6 Columns | PK: id

 Outgoing Foreign

Keys:

None (Root / Master Entity)

 Child Table

References:

No direct child tables



How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)



What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns:

id, email, otp, is_verified, expires_at, created_at

Domain 3: SaaS Plans, Subscriptions & PAYG Financials (14 Tables)

Governs software licensing tiers, recurring billing contracts, dynamic QR payment verifications, and itemized metered PAYG tax invoices.

saas_plans

19 Columns | PK: id

Outgoing Foreign

Keys:None (Root / Master Entity)

Child Table

References:saas_plan_analytics.plan_id saas_plan_limits.plan_id saas_plan_modules.plan_id saas_plan_pricing.plan_id saas_plan_versions.plan_id

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, name, code, description, long_description, icon, color, status, plan_type, pricing_model, payg_rules, currency, ...

saas_plan_limits

11 Columns | PK: id

Outgoing Foreign

Keys:plan_id → saas_plans.id

Child Table

References:No direct child tables

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, plan_id, max_users, max_locations, max_departments, max_projects, storage_limit_gb, api_limit, reports_limit, dashboards_limit, backups_limit

saas_plan_pricing

8 Columns | PK: id

Outgoing Foreign

Keys:plan_id → saas_plans.id

Child Table

References:No direct child tables

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, plan_id, billing_cycle, price, discount, tax, is_tax_inclusive, is_active

saas_plan_modules

5 Columns | PK: id

Outgoing Foreign

Keys:plan_id → saas_plans.id

Child Table

References:No direct child tables

How it Gets Data:User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored:Relational business entity attributes and metadata

Key Schema

Columns:id, plan_id, module_name, is_enabled, is_premium

 saas_plan_analytics

10 Columns | PK: id

 Outgoing Foreign**Keys:**

plan_id → saas_plans.id

 Child Table**References:***No direct child tables***How it Gets Data:****User Form Input via Web UI (Triggered by User Action via REST API)****What Data is Stored:**

Relational business entity attributes and metadata

 Key Schema**Columns:**

id, plan_id, mrr, arr, subscriber_count, renewal_rate, upgrade_rate, downgrade_rate, cancellation_rate, updated_at

 subscriptions

37 Columns | PK: id

 Outgoing Foreign**Keys:**

org_id → organizations.id created_by_id → users.id

 Child Table**References:**

subscription_invoices.subscription_id subscription_payments.subscription_id

**How it Gets Data:****User Form Input via Web UI (Triggered by User Action via REST API)****What Data is Stored:**

Relational business entity attributes and metadata

 Key Schema**Columns:**

id, org_id, subscription_uid, plan_name, pricing_model, payg_rules, billing_cycle, last_metered_billing_at, subscription_status, payment_status, start_date, end_date, ...

 subscription_invoices

27 Columns | PK: id

 Outgoing Foreign**Keys:**

payment_id → subscription_payments.id org_id → organizations.id subscription_id → subscriptions.id

 Child Table**References:**billing_audits.invoice_id invoice_items.invoice_id subscription_credit_notes.invoice_id
subscription_payments.invoice_id subscription_refunds.invoice_id**How it Gets Data:****'Generate Bill' click)****PAYG Billing Service / Subscription Renewal Engine (Triggered by Monthly cron cycle or manual****What Data is Stored:**

Itemized base fee, active seats, GB storage, 18% GST tax, invoice PDF binary reference

 Key Schema**Columns:**

id, subscription_id, org_id, invoice_uid, invoice_number, invoice_date, due_date, billing_period_start, billing_period_end, plan_name, billing_cycle, base_amount, ...

 invoice_items

6 Columns | PK: id

 Outgoing Foreign**Keys:**

invoice_id → subscription_invoices.id

 Child Table**References:***No direct child tables***How it Gets Data:****User Form Input via Web UI (Triggered by User Action via REST API)****What Data is Stored:**

Relational business entity attributes and metadata

 Key Schema**Columns:**

id, invoice_id, description, quantity, unit_price, amount

subscription_payments

22 Columns | PK: id

Outgoing Foreign

Keys:

invoice_id → subscription_invoices.id

org_id → organizations.id

subscription_id → subscriptions.id

Child Table

References:

subscription_invoices.payment_id

subscription_refunds.payment_id

How it Gets Data:

Razorpay Gateway Webhook / Super Admin QR Approval (Triggered by Payment completion or admin QR approval)

What Data is Stored: Realized monetary transaction ref, gateway payload, payment status, settlement date

Key Schema

Columns: id, org_id, subscription_id, invoice_id, amount, currency, plan_name, billing_cycle, payment_status, transaction_id, payment_gateway, gateway_reference, ...

subscription_refunds

8 Columns | PK: id

Outgoing Foreign

Keys:

payment_id → subscription_payments.id

invoice_id → subscription_invoices.id

Child Table

References:

No direct child tables

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, invoice_id, payment_id, refund_uid, amount, reason, status, created_at

offline_payment_proofs

15 Columns | PK: id

Outgoing Foreign

Keys:

org_id → organizations.id

verified_by_id → users.id

user_id → users.id

Child Table

References:

No direct child tables

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, org_id, user_id, plan_name, billing_cycle, amount, currency, transaction_id, screenshot_url, notes, status, rejection_reason, ...

billing_audits

8 Columns | PK: id

Outgoing Foreign

Keys:

user_id → users.id

org_id → organizations.id

invoice_id → subscription_invoices.id

Child Table

References:

No direct child tables

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, org_id, invoice_id, user_id, action, details, ip_address, created_at

tax_rules

6 Columns | PK: id

Outgoing Foreign

Keys:

None (Root / Master Entity)

Child Table

References:

No direct child tables

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, country, state, tax_type, rate, is_exempt

Domain 4: 8-Stage QC Quality Circle Workflow Engine (19 Tables)

The core operational backbone executing 8D DMAIC continuous quality problem-solving projects from Stage 1 Initiation to Stage 8 Closure.

projects

24 Columns | PK: id

🔗

Outgoing Foreign

Keys:

org_id → organizations.id

department_id → departments.id

reviewer_id → users.id

facilitator_id → users.id

creator_id → users.id

team_leader_id → users.id

📄

Child Table

References:

audit_logs.project_id

employee_points.project_id

facilitator_assistance_requests.project_id

facilitator_notes.project_id

imported_ideas.linked_project_id

knowledge_repository.project_id

kpi_metrics.project_id

project_meetings.project_id

+21 more

📄

How it Gets Data:

Team Leader / Quality Admin UI (Triggered by Project initialization form)

📦

What Data is Stored:

8-Stage project charter, problem description, assigned leader, facilitator, and reviewer

📄

Key Schema

Columns:

id, org_id, project_uid, title, description, creator_id, department_id, category, team_leader_id, facilitator_id, reviewer_id, deadline, ...

project_members

2 Columns | PK: project_id, user_id

🔗

Outgoing Foreign

Keys:

project_id → projects.id

user_id → users.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

📦

What Data is Stored:

Relational business entity attributes and metadata

📄

Key Schema

Columns:

project_id, user_id

project_reviews

10 Columns | PK: id

🔗

Outgoing Foreign

Keys:

org_id → organizations.id

project_id → projects.id

reviewer_id → users.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

📦

What Data is Stored:

Relational business entity attributes and metadata

📄

Key Schema

Columns:

id, org_id, project_id, stage_number, reviewer_id, status, decision, comments, decided_at, created_at

project_meetings

10 Columns | PK: id

🔗

Outgoing Foreign

Keys:

project_id → projects.id

org_id → organizations.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

📦

What Data is Stored:

Relational business entity attributes and metadata

📄

Key Schema

Columns:

id, org_id, project_id, stage_id, title, meeting_type, scheduled_at, duration, url, created_at

 project_workflow

9 Columns | PK: id

 Outgoing Foreign**Keys:**

project_id → projects.id

updated_by → users.id

org_id → organizations.id

 Child Table**References:***No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, project_id, stage_id, data, template_snapshot, completed_at, updated_by, updated_at

 facilitator_assistance_requests

11 Columns | PK: id

 Outgoing Foreign**Keys:**

facilitator_id → users.id

project_id → projects.id

org_id → organizations.id

user_id → users.id

 Child Table**References:***No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, project_id, stage_id, user_id, facilitator_id, message, status, response, created_at, updated_at

 facilitator_notes

7 Columns | PK: id

 Outgoing Foreign**Keys:**

created_by → users.id

org_id → organizations.id

project_id → projects.id

 Child Table**References:***No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, project_id, stage_number, note_text, created_by, created_at

 imported_ideas

21 Columns | PK: id

 Outgoing Foreign**Keys:**

organization_id → organizations.id

linked_project_id → projects.id

 Child Table**References:***No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, organization_id, idea_code, title, problem_statement, proposed_solution, department, category, submitted_by, co_suggesters, tangible_benefit, intangible_benefit, ...

 kpi_metrics

8 Columns | PK: id

 Outgoing Foreign**Keys:**

project_id → projects.id

org_id → organizations.id

 Child Table**References:***No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, project_id, cost_saving, productivity_gain, quality_index, safety_score, updated_at

stage_1_problem_definition_project_initiation

15 Columns | PK: id

Outgoing Foreign

Keys:

project_id → projects.id

facilitator_approver_id → users.id

org_id → organizations.id

Child Table

References:

No direct child tables

How it Gets Data:

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

What Data is Stored:

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

Key Schema

Columns:

id, org_id, project_id, project_team, problem_5w2h, current_performance, justification, emergency_response, theme_target_schedule, facilitator_approved, facilitator_approver_id, facilitator_approved_at, ...

stage_2_observation_data_collection

15 Columns | PK: id

Outgoing Foreign

Keys:

project_id → projects.id

org_id → organizations.id

reviewer_id → users.id

Child Table

References:

No direct child tables

How it Gets Data:

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

What Data is Stored:

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

Key Schema

Columns:

id, org_id, project_id, process_observation, standard_verification, data_collection, stratification, pareto, five_g, current_state, reviewer_approved, reviewer_id, ...

stage_4_root_cause_analysis_verification

16 Columns | PK: id

Outgoing Foreign

Keys:

reviewer_id → users.id

project_id → projects.id

org_id → organizations.id

Child Table

References:

No direct child tables

How it Gets Data:

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

What Data is Stored:

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

Key Schema

Columns:

id, org_id, project_id, verified_causes, hypothesis_testing, good_vs_bad, statistical_validation, data_reconfirmation, why_why_analysis, root_cause_register, root_cause_ranking, reviewer_approved, ...

stage_5_countermeasure_planning_solution_development

26 Columns | PK: id

Outgoing Foreign

Keys:

facilitator_id → users.id

reviewer_id → users.id

project_id → projects.id

org_id → organizations.id

Child Table

References:

No direct child tables

How it Gets Data:

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

What Data is Stored:

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

Key Schema

Columns:

id, org_id, project_id, root_cause_mapping, solution_brainstorming, solution_evaluation, cost_benefit_analysis, side_effect_analysis, pilot_solution_verification, action_plan_3w1h, resource_planning, fishbone_data, ...

 stage_6_implementation_change_management

18 Columns | PK: id

 **Outgoing Foreign**

Keys:

reviewer_id → users.id

project_id → projects.id

org_id → organizations.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

 **What Data is Stored:**

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

 **Key Schema**

Columns: id, org_id, project_id, implementation_execution, task_management, resource_planning_deployment, resource_deployment, change_management, risk_resistance, training_awareness, communication_log, implementation_evidence, ...

 stage_7_performance_verification_benefits_realization

18 Columns | PK: id

 **Outgoing Foreign**

Keys:

org_id → organizations.id

reviewer_id → users.id

project_id → projects.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

 **What Data is Stored:**

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

 **Key Schema**

Columns: id, org_id, project_id, kpi_verification, before_vs_after, statistical_validation, benefit_realization, roi_validation, sustainability_check, side_effect_verification, lessons_implementation, action_plan, ...

 stage_8_standardization_knowledge_sharing_project_closure

32 Columns | PK: id

 **Outgoing Foreign**

Keys:

final_approval_by → users.id

project_id → projects.id

org_id → organizations.id

approved_by → users.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

Quality Circle Team Project Workstation (Triggered by Step save or 'Submit Stage for Review' action)

 **What Data is Stored:**

Stage-specific problem statements, root cause 5-Why trees, countermeasure matrices, signoff stamps

 **Key Schema**

Columns: id, org_id, project_id, standardization, training_adoption, horizontal_deployment, lessons_learned, benefits_summary, remaining_opportunities, knowledge_repository, team_recognition, project_closure, ...

Domain 5: 7 QC Tools & Statistical Process Control (15 Tables)

Captures raw shop-floor quality data and renders Ishikawa 6M fishbone trees, Pareto distributions, SPC control charts, check sheets, and scatter correlations.

qc_fishbone_diagrams

6 Columns | PK: id

Outgoing Foreign

Keys:project_id → projects.idorg_id → organizations.id

Child Table

References:qc_fishbone_branches.fishbone_id

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

Key Schema

Columns:id, org_id, project_id, stage_id, effect, created_at

qc_fishbone_branches

5 Columns | PK: id

Outgoing Foreign

Keys:parent_cause_id → qc_fishbone_branches.idfishbone_id → qc_fishbone_diagrams.id

Child Table

References:qc_fishbone_branches.parent_cause_id

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

Key Schema

Columns:id, fishbone_id, category, parent_cause_id, text

qc_pareto_charts

8 Columns | PK: id

Outgoing Foreign

Keys:org_id → organizations.idproject_id → projects.id

Child Table

References:qc_pareto_items.pareto_chart_id

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

Key Schema

Columns:id, org_id, project_id, stage_id, title, description, total_count, created_at

qc_pareto_items

6 Columns | PK: id

Outgoing Foreign

Keys:pareto_chart_id → qc_pareto_charts.id

Child Table

References:No direct child tables

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

Key Schema

Columns:id, pareto_chart_id, cause_name, frequency, cumulative_pct, priority_rank

 qc_control_charts

11 Columns | PK: id

 Outgoing Foreign**Keys:**[project_id → projects.id](#) [org_id → organizations.id](#) **Child Table****References:**[qc_control_points.control_chart_id](#) **How it Gets Data:****Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)** **What Data is Stored:** Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies **Key Schema****Columns:**

id, org_id, project_id, stage_id, title, chart_type, mean, ucl, lcl, std_dev, created_at

 qc_control_points

5 Columns | PK: id

 Outgoing Foreign**Keys:**[control_chart_id → qc_control_charts.id](#) **Child Table****References:***No direct child tables* **How it Gets Data:****Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)** **What Data is Stored:** Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies **Key Schema****Columns:**

id, control_chart_id, sample_index, value, is_out_of_control

 qc_check_sheets

7 Columns | PK: id

 Outgoing Foreign**Keys:**[project_id → projects.id](#) [org_id → organizations.id](#) **Child Table****References:**[qc_check_sheet_entries.check_sheet_id](#) [qc_check_sheet_rows.check_sheet_id](#) **How it Gets Data:****Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)** **What Data is Stored:** Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies **Key Schema****Columns:**

id, org_id, project_id, stage_id, name, description, created_at

 qc_check_sheet_rows

5 Columns | PK: id

 Outgoing Foreign**Keys:**[check_sheet_id → qc_check_sheets.id](#) **Child Table****References:**[qc_check_sheet_entries.row_id](#) **How it Gets Data:****Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)** **What Data is Stored:** Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies **Key Schema****Columns:**

id, check_sheet_id, category_name, total_count, notes

 qc_check_sheet_entries

7 Columns | PK: id

 Outgoing Foreign**Keys:**[check_sheet_id → qc_check_sheets.id](#) [row_id → qc_check_sheet_rows.id](#) **Child Table****References:***No direct child tables* **How it Gets Data:****Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)** **What Data is Stored:** Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies **Key Schema****Columns:**

id, check_sheet_id, row_id, date, count, inspector, remarks

qc_scatter_diagrams

9 Columns | PK: id

🔗

Outgoing Foreign

Keys:

org_id → organizations.id

project_id → projects.id

📄

Child Table

References:

qc_scatter_points.scatter_diagram_id

📄

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

📄

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

📄

Key Schema

Columns:

id, org_id, project_id, stage_id, x_axis_label, y_axis_label, correlation_coefficient, correlation_type, created_at

qc_scatter_points

5 Columns | PK: id

🔗

Outgoing Foreign

Keys:

scatter_diagram_id → qc_scatter_diagrams.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

📄

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

📄

Key Schema

Columns:

id, scatter_diagram_id, x_value, y_value, remarks

qc_stratifications

8 Columns | PK: id

🔗

Outgoing Foreign

Keys:

org_id → organizations.id

project_id → projects.id

📄

Child Table

References:

qc_stratification_items.stratification_id

📄

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

📄

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

📄

Key Schema

Columns:

id, org_id, project_id, stage_id, title, description, category_type, created_at

qc_stratification_items

5 Columns | PK: id

🔗

Outgoing Foreign

Keys:

stratification_id → qc_stratifications.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

📄

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

📄

Key Schema

Columns:

id, stratification_id, factor_name, defect_count, percentage

qc_process_maps

7 Columns | PK: id

🔗

Outgoing Foreign

Keys:

project_id → projects.id

org_id → organizations.id

📄

Child Table

References:

qc_process_steps.process_map_id

📄

How it Gets Data:

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

📄

What Data is Stored:

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

📄

Key Schema

Columns:

id, org_id, project_id, stage_id, title, description, created_at

 **qc_process_steps**

7 Columns | PK: id

 **Outgoing Foreign**

Keys:

process_map_id → qc_process_maps.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

Interactive 7 QC Tool Workspace (Triggered by QC tool editor save / import)

 **What Data is Stored:**

Fishbone 6M nodes, Pareto defect frequencies, SPC X-bar/R-chart limits, check sheet tallies

 **Key Schema**

Columns: id, process_map_id, step_order, name, type, description, next_step_id

Domain 6: Standard Operating Procedures (SOP) & Training (13 Tables)

Standardizes controlled procedures, tracks multi-tier change revisions, administers mandatory workforce training, and logs comprehension assessments.

sop_master

35 Columns | PK: id

🔗

Outgoing Foreign

Keys:

approver_id → users.id

reviewer_id → users.id

author_id → users.id

department_id → departments.id

owner_id → users.id

org_id → organizations.id

project_id → projects.id

📄

Child Table

References:

assessment_questions.sop_id

sop_approvals.sop_id

sop_comments.sop_id

sop_steps.sop_id

sop_versions.sop_id

training_archive.sop_id

training_assessments.sop_id

training_assignments.sop_id

📄

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

📦

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

📄

Key Schema

Columns:

id, sop_uid, org_id, title, category, department_id, process_name, sop_type, description, purpose, scope, applicability, ...

sop_steps

9 Columns | PK: id

🔗

Outgoing Foreign

Keys:

sop_id → sop_master.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

📦

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

📄

Key Schema

Columns:

id, sop_id, step_number, step_title, instructions, image_path, video_path, safety_notes, quality_checkpoints

sop_versions

8 Columns | PK: id

🔗

Outgoing Foreign

Keys:

sop_id → sop_master.id

changed_by_id → users.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

📦

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

📄

Key Schema

Columns:

id, sop_id, version_number, changes_made, changed_by_id, changed_date, approval_date, sop_data

sop_approvals

8 Columns | PK: id

🔗

Outgoing Foreign

Keys:

sop_id → sop_master.id

user_id → users.id

📄

Child Table

References:

No direct child tables

📄

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

📦

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

📄

Key Schema

Columns:

id, sop_id, user_id, role, action, comments, signature, created_at

Page 28

sop_comments

7 Columns | PK: id

Outgoing Foreign Keys:

user_id → users.id

sop_id → sop_master.id

Child Table

References: *No direct child tables*

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

Key Schema

Columns: id, sop_id, user_id, role, comment_type, content, created_at

sop_categories

5 Columns | PK: id

Outgoing Foreign Keys:

org_id → organizations.id

Child Table

References: *No direct child tables*

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

Key Schema

Columns: id, org_id, name, description, created_at

sop_types

5 Columns | PK: id

Outgoing Foreign Keys:

org_id → organizations.id

Child Table

References: *No direct child tables*

How it Gets Data:

Standard Operating Procedure Document Manager (Triggered by SOP creation, revision edit, or approval click)

What Data is Stored:

Standardized operating steps, hazard warnings, QC check limits, revision history, approval signatures

Key Schema

Columns: id, org_id, name, description, created_at

training_assignments

18 Columns | PK: id

Outgoing Foreign Keys:

sop_id → sop_master.id

assigned_by_id → users.id

user_id → users.id

Child Table

References:

assessment_results.training_id

training_acknowledgements.training_id

training_certificates.training_id

training_notifications.training_id

How it Gets Data:

Workforce Training & Assessment Module (Triggered by Training assignment or employee quiz submission)

What Data is Stored:

Employee training completion status, quiz assessment scores, comprehension certificates

Key Schema

Columns: id, sop_id, user_id, assigned_date, read_status, acknowledgement_status, training_completion_status, assessment_score, completed_at, assigned_by_id, due_date, expiry_date, ...

 assessment_results

9 Columns | PK: id

 Outgoing Foreign**Keys:**[training_id → training_assignments.id](#) [user_id → users.id](#) **Child Table****References:***No direct child tables***How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)****What Data is Stored:** Relational business entity attributes and metadata**Key Schema****Columns:**

id, training_id, user_id, score, percentage, attempt_number, result, answers_submitted, created_at

 training_acknowledgements

7 Columns | PK: id

 Outgoing Foreign**Keys:**[user_id → users.id](#) [training_id → training_assignments.id](#) **Child Table****References:***No direct child tables***How it Gets Data:** **Workforce Training & Assessment Module (Triggered by Training assignment or employee quiz submission)****What Data is Stored:** Employee training completion status, quiz assessment scores, comprehension certificates**Key Schema****Columns:**

id, training_id, user_id, statement, ip_address, digital_signature, created_at

 training_notifications

8 Columns | PK: id

 Outgoing Foreign**Keys:**[training_id → training_assignments.id](#) [user_id → users.id](#) **Child Table****References:***No direct child tables***How it Gets Data:** **Workforce Training & Assessment Module (Triggered by Training assignment or employee quiz submission)****What Data is Stored:** Employee training completion status, quiz assessment scores, comprehension certificates**Key Schema****Columns:**

id, training_id, user_id, notification_type, message, is_sent, sent_at, created_at

 training_audit_reports

7 Columns | PK: id

 Outgoing Foreign**Keys:**[project_id → projects.id](#) [org_id → organizations.id](#) [generated_by_id → users.id](#) **Child Table****References:***No direct child tables***How it Gets Data:** **Workforce Training & Assessment Module (Triggered by Training assignment or employee quiz submission)****What Data is Stored:** Employee training completion status, quiz assessment scores, comprehension certificates**Key Schema****Columns:**

id, org_id, project_id, generated_by_id, report_type, pdf_path, created_at

 training_archive

5 Columns | PK: id

 **Outgoing Foreign**

Keys: archived_by_id → users.id sop_id → sop_master.id

 **Child Table**

References: No direct child tables

 **How it Gets Data:**

Workforce Training & Assessment Module (Triggered by Training assignment or employee quiz submission)

 **What Data is Stored:**

Employee training completion status, quiz assessment scores, comprehension certificates

 **Key Schema**

Columns: id, sop_id, archived_by_id, archived_at, reason

Domain 7: Dynamic Enterprise Modules & Feature Gating (9 Tables)

Controls hierarchical module trees, version releases, dependencies, and per-tenant feature access enablement.

modules 39 Columns PK: id Outgoing Foreign Keys: parent_id → modules.idcreated_by_id → users.id Child Table References: feature_versions.module_idmodule_analytics.module_idmodule_assignments.module_idmodule_audit_logs.module_idmodule_dependencies.dependency_module_idmodule_dependencies.module_idmodule_permissions.module_idmodules.parent_id+1 more How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, parent_id, name, code, description, category, icon, color, display_order, navigation_route, api_prefix, status, ...
organization_features 8 Columns PK: id Outgoing Foreign Keys: org_id → organizations.idmodule_id → modules.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, org_id, module_id, is_enabled, notes, expires_at, created_at, updated_at
module_dependencies 4 Columns PK: id Outgoing Foreign Keys: dependency_module_id → modules.idmodule_id → modules.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, module_id, dependency_module_id, dependency_type
module_permissions 4 Columns PK: id Outgoing Foreign Keys: module_id → modules.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, module_id, role_name, permissions

module_assignments

5 Columns | PK: id

Outgoing Foreign

Keys: [module_id → modules.id](#)

Child Table

References: *No direct child tables*

How it Gets Data: **User Form Input via Web UI (Triggered by User Action via REST API)**

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, module_id, assigned_type, assigned_target, assignment_metadata

feature_versions

6 Columns | PK: id

Outgoing Foreign

Keys: [created_by_id → users.id](#) [module_id → modules.id](#)

Child Table

References: *No direct child tables*

How it Gets Data: **User Form Input via Web UI (Triggered by User Action via REST API)**

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, module_id, version_number, snapshot_json, created_by_id, created_at

module_analytics

11 Columns | PK: id

Outgoing Foreign

Keys: [org_id → organizations.id](#) [module_id → modules.id](#)

Child Table

References: *No direct child tables*

How it Gets Data: **User Form Input via Web UI (Triggered by User Action via REST API)**

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, module_id, org_id, active_users, daily_usage, monthly_usage, api_calls, storage_consumption_mb, performance_ms, error_rate, timestamp

module_audit_logs

6 Columns | PK: id

Outgoing Foreign

Keys: [module_id → modules.id](#)

Child Table

References: *No direct child tables*

How it Gets Data: **User Form Input via Web UI (Triggered by User Action via REST API)**

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, module_id, admin_name, action, details, timestamp

feature_categories

7 Columns | PK: id

Outgoing Foreign

Keys: *None (Root / Master Entity)*

Child Table

References: *No direct child tables*

How it Gets Data: **User Form Input via Web UI (Triggered by User Action via REST API)**

What Data is Stored: Relational business entity attributes and metadata

Key Schema

Columns: id, name, code, description, icon, display_order, created_at

Domain 8: Analytics, AI Insights & BI Reporting (7 Tables)

Generates platform-wide BI charts, AI predictive anomaly alerts, automated schedule deliveries, and Excel/PDF report exports.

analytics_reports 7 Columns PK: id Outgoing Foreign Keys: created_by_id → users.id org_id → organizations.id Child Table References: analytics_schedules.report_id How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, org_id, title, description, config_json, created_by_id, created_at
analytics_schedules 10 Columns PK: id Outgoing Foreign Keys: org_id → organizations.id report_id → analytics_reports.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, org_id, report_id, frequency, format, recipient_emails, next_run, last_run, is_active, created_at
analytics_usage 7 Columns PK: id Outgoing Foreign Keys: user_id → users.id org_id → organizations.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, org_id, user_id, module_name, feature_name, action_type, created_at
analytics_exports 8 Columns PK: id Outgoing Foreign Keys: user_id → users.id org_id → organizations.id Child Table References: No direct child tables How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) What Data is Stored: Relational business entity attributes and metadata Key Schema Columns: id, org_id, user_id, report_type, format, file_path, filters_applied, generated_at

analytics_ai_insights 6 Columns PK: id
🔗 Outgoing Foreign Keys: org_id → organizations.id 📄 Child Table References: No direct child tables 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: id, org_id, insight_type, insight_json, recommendation, generated_at
kpi_dashboard_cache 5 Columns PK: id
🔗 Outgoing Foreign Keys: org_id → organizations.id 📄 Child Table References: No direct child tables 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: id, org_id, cache_key, data, updated_at
analytics_cache 5 Columns PK: id
🔗 Outgoing Foreign Keys: None (Root / Master Entity) 📄 Child Table References: No direct child tables 📄 How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API) 📦 What Data is Stored: Relational business entity attributes and metadata 📄 Key Schema Columns: id, cache_key, cache_data, expires_at, created_at

Domain 9: Helpdesk Support, SLAs & Sales Prospect Intake (7 Tables)

Tracks customer support tickets, priority SLAs, internal collaboration threads, and landing page prospect leads.

 support_tickets	17 Columns PK: id
Outgoing Foreign Keys: org_id → organizations.id assigned_engineer_id → users.id user_id → users.id	
Child Table References: support_attachments.ticket_id support_audits.ticket_id support_comments.ticket_id support_escalations.ticket_id support_ratings.ticket_id support_slas.ticket_id	
How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API)	
What Data is Stored: Relational business entity attributes and metadata	
Key Schema Columns: id, org_id, user_id, subject, message, priority, status, category, created_at, resolved_at, resolution, ticket_number, ...	
 support_comments	7 Columns PK: id
Outgoing Foreign Keys: user_id → users.id ticket_id → support_tickets.id	
Child Table References: support_attachments.comment_id	
How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API)	
What Data is Stored: Relational business entity attributes and metadata	
Key Schema Columns: id, ticket_id, user_id, content, is_internal, created_at, updated_at	
 support_attachments	10 Columns PK: id
Outgoing Foreign Keys: comment_id → support_comments.id ticket_id → support_tickets.id uploaded_by_id → users.id	
Child Table References: <i>No direct child tables</i>	
How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API)	
What Data is Stored: Relational business entity attributes and metadata	
Key Schema Columns: id, ticket_id, comment_id, file_name, file_path, file_size, mime_type, uploaded_by_id, uploaded_at, virus_scan_passed	
 support_audits	7 Columns PK: id
Outgoing Foreign Keys: user_id → users.id ticket_id → support_tickets.id	
Child Table References: <i>No direct child tables</i>	
How it Gets Data: User Form Input via Web UI (Triggered by User Action via REST API)	
What Data is Stored: Relational business entity attributes and metadata	
Key Schema Columns: id, ticket_id, user_id, action, old_values, new_values, created_at	

 support_ratings

5 Columns | PK: id

 Outgoing Foreign

Keys: ticket_id → support_tickets.id

 Child Table

References: No direct child tables

 How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

 What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns: id, ticket_id, rating, feedback, created_at

 support_knowledge

8 Columns | PK: id

 Outgoing Foreign

Keys: created_by_id → users.id

 Child Table

References: No direct child tables

 How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

 What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns: id, title, category, content, is_internal, created_by_id, created_at, views_count

 sales_enquiries

11 Columns | PK: id

 Outgoing Foreign

Keys: None (Root / Master Entity)

 Child Table

References: No direct child tables

 How it Gets Data:

User Form Input via Web UI (Triggered by User Action via REST API)

 What Data is Stored:

Relational business entity attributes and metadata

 Key Schema

Columns: id, name, email, phone, company_name, message, source, status, notes, created_at, updated_at

Domain 10: Communications, Announcements, Integrations & Governance (26 Tables)

Dispatches tenant broadcast announcements, manages outbound webhooks, logs immutable security audit events, and maintains global platform settings.

announcements 24 Columns PK: id Outgoing Foreign Keys: created_by → users.id org_id → organizations.id Child Table References: announcement_attachments.announcement_id announcement_audience.announcement_id announcement_audit.announcement_id announcement_delivery.announcement_id announcement_reads.announcement_id How it Gets Data: Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) What Data is Stored: Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue Key Schema Columns: id, ann_number, org_id, created_by, title, summary, body, banner_url, tags, category, priority, status, ...
announcement_delivery 10 Columns PK: id Outgoing Foreign Keys: user_id → users.id announcement_id → announcements.id org_id → organizations.id Child Table References: No direct child tables How it Gets Data: Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) What Data is Stored: Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue Key Schema Columns: id, announcement_id, org_id, user_id, channel, status, sent_at, error_message, retry_count, created_at
announcement_reads 12 Columns PK: id Outgoing Foreign Keys: user_id → users.id announcement_id → announcements.id org_id → organizations.id Child Table References: No direct child tables How it Gets Data: Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) What Data is Stored: Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue Key Schema Columns: id, announcement_id, user_id, org_id, viewed_at, read_at, clicked_at, dismissed_at, device, browser, ip_address, created_at
announcement_attachments 8 Columns PK: id Outgoing Foreign Keys: uploaded_by → users.id announcement_id → announcements.id Child Table References: No direct child tables How it Gets Data: Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) What Data is Stored: Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue Key Schema Columns: id, announcement_id, file_name, file_url, file_size, file_type, uploaded_by, created_at

 announcement_audit

7 Columns | PK: id

 Outgoing Foreign**Keys:**

announcement_id → announcements.id

user_id → users.id

 Child Table**References:***No direct child tables* **How it Gets Data:** Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) **What Data is Stored:** Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue **Key Schema****Columns:**

id, announcement_id, user_id, action, details, ip_address, created_at

 announcement_audience

4 Columns | PK: id

 Outgoing Foreign**Keys:**

announcement_id → announcements.id

 Child Table**References:***No direct child tables* **How it Gets Data:** Super Admin Broadcast Console (Triggered by Admin 'Broadcast Now' action) **What Data is Stored:** Broadcast title, rich-text body, audience filter tags, per-user read receipts, delivery queue **Key Schema****Columns:**

id, announcement_id, target_type, target_value

 email_notification_rules

30 Columns | PK: id

 Outgoing Foreign**Keys:**

created_by_id → users.id

 Child Table**References:**

email_notification_logs.rule_id

 **How it Gets Data:** User Form Input via Web UI (Triggered by User Action via REST API) **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, name, category, description, subject, preheader, heading, body_html, banner_color, cta_text, cta_url, sender_email, ...

 email_notification_logs

13 Columns | PK: id

 Outgoing Foreign**Keys:**

sent_by_id → users.id

rule_id → email_notification_rules.id

 Child Table**References:***No direct child tables* **How it Gets Data:** User Form Input via Web UI (Triggered by User Action via REST API) **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, rule_id, rule_name, category, subject, sender_email, sender_name, recipient_count, recipients_summary, status, error_message, sent_by_id, ...

 notifications

9 Columns | PK: id

 Outgoing Foreign**Keys:**

user_id → users.id

org_id → organizations.id

 Child Table**References:***No direct child tables* **How it Gets Data:** User Form Input via Web UI (Triggered by User Action via REST API) **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, user_id, title, message, is_read, is_starred, created_at, link

 audit_logs

24 Columns | PK: id

 Outgoing Foreign**Keys:**

user_id → users.id

org_id → organizations.id

project_id → projects.id

 Child Table**References:**

audit_risk_alerts.log_id

 How it Gets Data:**Platform Security Interceptor (Triggered by Automatic event listener on every database mutation)** **What Data is Stored:** Immutable trace: Client IP, user ID, tenant ID, action name, old payload JSON, new payload JSON **Key Schema****Columns:**

id, org_id, project_id, user_id, action, details, ip_address, user_agent, target_table, target_id, created_at, session_id, ...

 audit_export_logs

7 Columns | PK: id

 Outgoing Foreign**Keys:**

user_id → users.id

org_id → organizations.id

 Child Table**References:**

No direct child tables

 How it Gets Data:**User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, org_id, user_id, export_type, record_count, details, created_at

 super_admin_logs

8 Columns | PK: id

 Outgoing Foreign**Keys:**

admin_id → users.id

 Child Table**References:**

No direct child tables

 How it Gets Data:**User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, admin_id, action, target_type, target_id, details, ip_address, created_at

 integration_webhooks

8 Columns | PK: id

 Outgoing Foreign**Keys:**

None (Root / Master Entity)

 Child Table**References:**

integration_webhook_deliveries.webhook_id

 How it Gets Data:**User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, name, url, secret, status, headers, events, created_at

 integration_webhook_deliveries

10 Columns | PK: id

 Outgoing Foreign**Keys:**

webhook_id → integration_webhooks.id

 Child Table**References:**

No direct child tables

 How it Gets Data:**User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:**

id, webhook_id, event, payload, response_code, response_body, latency_ms, retry_count, status, created_at

 integration_api_logs

9 Columns | PK: id

 Outgoing Foreign**Keys:** [organization_id → organizations.id](#) **Child Table****References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:** id, organization_id, ip_address, api_key_used, request_time, response_time_ms, endpoint, status_code, created_at **integration_api_keys**

15 Columns | PK: id

 Outgoing Foreign**Keys:** *None (Root / Master Entity)* **Child Table****References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:** id, name, key_prefix, api_key_hash, secret_key_masked, status, expiration_date, rate_limit, allowed_ips, allowed_domains, scopes, owner, ... **org_api_keys**

10 Columns | PK: id

 Outgoing Foreign**Keys:** [organization_id → organizations.id](#) **Child Table****References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:** id, organization_id, api_key_hash, key_prefix, secret_key_masked, status, usage_count, last_used, created_at, updated_at **integration_configs**

12 Columns | PK: id

 Outgoing Foreign**Keys:** *None (Root / Master Entity)* **Child Table****References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:** id, provider_id, provider_name, category, status, version, settings, health_score, last_sync, usage_count, error_count, updated_at **integration_audit_logs**

5 Columns | PK: id

 Outgoing Foreign**Keys:** *None (Root / Master Entity)* **Child Table****References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata **Key Schema****Columns:** id, action, provider_id, details, created_at

 platform_settings

54 Columns | PK: id

 Outgoing Foreign**Keys:** *None (Root / Master Entity)* Child Table**References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata Key Schema**Columns:** id, site_name, maintenance_mode, registration_open, require_email_otp, require_phone_otp, global_notification, support_email, system_version, global_template_version, global_template_updated_at, global_template_release_notes, ... document_templates

17 Columns | PK: id

 Outgoing Foreign**Keys:** [org_id → organizations.id](#) Child Table**References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata Key Schema**Columns:** id, org_id, template_key, template_name, header_title, subtitle, header_text, footer_text, watermark_text, confidential_text, terms_and_conditions, disclaimer_text, ... setting_usage_map

17 Columns | PK: id

 Outgoing Foreign**Keys:** *None (Root / Master Entity)* Child Table**References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata Key Schema**Columns:** id, setting_key, setting_name, module, feature, component, page, route, backend_service, document_type, template, export_type, ... compliance_standard_records

22 Columns | PK: id

 Outgoing Foreign**Keys:** [org_id → organizations.id](#) Child Table**References:** *No direct child tables* **How it Gets Data:** **User Form Input via Web UI (Triggered by User Action via REST API)** **What Data is Stored:** Relational business entity attributes and metadata Key Schema**Columns:** id, org_id, standard_name, standard_code, description, icon, is_enabled, certificate_number, issue_date, expiry_date, last_audit_date, next_audit_date, ...

 audit_risk_alerts

10 Columns | PK: id

 **Outgoing Foreign**

Keys:

org_id → organizations.id

user_id → users.id

log_id → audit_logs.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

User Form Input via Web UI (Triggered by User Action via REST API)

 **What Data is Stored:**

 Relational business entity attributes and metadata

 **Key Schema**

Columns: id, org_id, user_id, log_id, risk_score, risk_level, triggered_rules, suggested_actions, status, created_at

 knowledge_repository

18 Columns | PK: id

 **Outgoing Foreign**

Keys:

department_id → departments.id

org_id → organizations.id

project_id → projects.id

 **Child Table**

References: *No direct child tables*

 **How it Gets Data:**

User Form Input via Web UI (Triggered by User Action via REST API)

 **What Data is Stored:**

 Relational business entity attributes and metadata

 **Key Schema**

Columns: id, org_id, project_id, title, department_id, category, problem_summary, root_cause, solution_summary, kpi_improvement_pct, cost_savings, sop_path, ...